INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM

(An Institute of National Importance)

IOE-414: DEEP LEARNING

MINOR PROJECT REPORT

Project Title: Reproducing and Extending GAF-Net for Multi-Disease Corn Leaf Detection

Group Members:

1. Maram Sai Manideep (2023BCS0095)
2. Mannava Venkata Raghupathi Sai (2023BCS0096)

Department of Computer Science and Engineering

October–2026

Table of Contents

Table of Contents **2**

Abstract **3**

Problem Statement **3**

Method Overview with Architecture **4**

Baseline architecture 4

GSConv and FASFF 5

Our modifications 5

Ablation design 7

Dataset Details **8**

Results and Visualizations **9**

Training configuration 9

An uncontrolled variable in the first comparison 9

Results on the test split 10

Discussion 10

Limitations 11

Implementation and Dataset Links [GitHub] **11**

References [APA Style] **12**

Abstract

GAF-Net is a YOLOv8-based detector for corn leaf pests and diseases. It adds two modules to the neck of the network: GSConv, a lightweight convolution, and FASFF, a block that learns how much to trust each level of the feature pyramid at every pixel. Its authors report 80.8% mAP@0.5 on a dataset they collected themselves, which is 4.1 points above their YOLOv8 baseline. We reimplemented both modules and tested them on a public corn leaf dataset of 10,884 images across four classes.

Our first comparison suggested that GAF-Net was performing worse than the plain baseline, with most of the loss concentrated in the gray spot class. The cause turned out not to be the architecture. The two runs had been launched with different optimizer settings, so the comparison was never controlled. Once every model was retrained under one identical recipe, GAF-Net matched the baseline on mAP@0.5 (0.9836 against 0.9833), improved mAP@0.5:0.95 by 0.7 points, and did so with 36% fewer parameters and 20% fewer FLOPs.

We then tested two modifications of our own. Correcting the downsampling path inside GSConv gave the best mAP@0.5 of the four models, though by a margin too small to claim. A cross-level, channel-grouped fusion gate intended to replace FASFF did not help and in fact scored lowest. We relate this to the dataset, which carries one large object per image and leaves only 1.7 points of headroom above a baseline already at 98.3%.

Base Paper Details

- **Base Paper Title:** GAF-Net: An Enhanced Multidisease for Corn Leaves Detection Model Based on YOLOv8
- **DOI link of the paper:** https://doi.org/10.1109/TAFE.2025.3561196
- **Conference/Journal Name:** IEEE Transactions on AgriFood Electronics, Vol. 3, No. 2
- **Authors:** Kai Cao, Yujing Yang, Li Xu, Xiaolong Wan, Liangliang Yang and Weikuan Jia
- **Year of Publication:** 2025

Problem Statement

Corn is among the most important cereal crops grown in India, and leaf diseases such as northern leaf blight, common rust and gray leaf spot cut yields every season. Diagnosis in the field is still mostly manual. A farmer or an extension officer inspects leaves by eye, which is slow across a large plot and depends heavily on experience. Two diseases that

look similar in their early stages are easy to confuse, and treating for the wrong one wastes both chemicals and time.

Automating this is usually framed as image classification: feed in a photograph of a leaf and get back a disease name. That is useful but limited. A single leaf can carry more than one infection, and knowing where the lesions sit matters for judging severity and for targeting a spray. Object detection answers both questions at once, which is why recent work in this area has moved towards detectors rather than classifiers.

The difficulty is that detectors built for general-purpose benchmarks do not transfer cleanly to leaves. Lesions vary enormously in size, from rust pustules a few pixels across to blight streaks running most of the length of a leaf. Different diseases share colour and texture, so the gap between classes is small. Field photographs bring their own problems with shadow, overlapping leaves and cluttered soil backgrounds. On top of that, a model meant for use in a field has to run on modest hardware, so accuracy cannot be bought with an arbitrarily large network.

GAF-Net (Cao et al., 2025) addresses this by modifying the neck of YOLOv8 in two places and reports a 4.1 point gain in mAP@0.5 over its own baseline. That result is specific to the dataset the authors built. The question we set out to answer in this project was whether the same two modules give a comparable benefit on a different, larger, publicly available corn leaf dataset, and if not, why not. A second question emerged during the work: what has to be held constant for a comparison between two detector variants to mean anything at all.

Method Overview with Architecture

Baseline architecture

All four models in this study share the same skeleton. YOLOv8 takes a 640 by 640 image and passes it through three stages. The backbone extracts features and produces three feature maps at different resolutions, written P3, P4 and P5, which have been downsampled by factors of 8, 16 and 32. P3 is the largest and carries fine texture; P5 is the smallest and carries coarse semantic information. The neck mixes these three maps together, first top-down and then bottom-up, following the PANet pattern. The head predicts a bounding box and a class for each position.

The main repeated block in the backbone is C2f. It splits its input in half, sends one half through a series of bottleneck blocks, and concatenates the untouched half with the output of every bottleneck along the way. Because the shallow and deep representations are both carried forward, gradients reach the early layers without passing through the whole stack. The idea comes from CSPNet (Wang et al., 2020), and C2f is the version used in YOLOv8. We mention it here because it becomes relevant later: our implementation of

GAF-Net removed every C2f from the neck, and one of our modifications puts a single one back.

GSConv and FASFF

GSConv (Li, Li, et al., 2024) is a cheaper substitute for a standard convolution. It runs a normal convolution that produces half the required channels, passes that result through a depthwise convolution to produce the other half, concatenates the two, and then shuffles the channels so that the next layer sees them interleaved rather than in two separate blocks. The shuffle is not cosmetic; without it the two paths never actually mix. GAF-Net uses GSConv only in the neck, on the grounds that using it throughout would lengthen inference.

FASFF is built on ASFF (Liu et al., 2019) and decides how much of each pyramid level to use. For each output level, the three feature maps are resized to a common resolution and reduced to a common channel count. Each one is then passed through its own 1 by 1 convolution that produces a single channel, a spatial map of one score per pixel. A softmax across the three levels normalises the scores, and the fused output is the weighted sum. Two details of this scheme matter for what follows. Each level computes its score from itself alone, without seeing the other two, and the score is a single number per pixel shared by every channel.

Our modifications

The first change concerns how GSConv reduces resolution. In our initial implementation the strided convolution used a 1 by 1 kernel, so at stride 2 it read one pixel out of every four and discarded the rest with no spatial support at all. This is a straightforward source of aliasing, and it affects fine texture more than anything else (Zhang, 2019). It also departs from the GSConv paper, whose first equation specifies a k by k kernel. We replaced it with a 3 by 3 strided convolution and added the batch normalisation and activation that branch was missing. We call the corrected version GSConvX. Alongside it we restored one C2f block at the P3 branch of the neck, where the highest-resolution features are, and left GSConv in place elsewhere.

The second change is a replacement for FASFF, which we call CG-FASFF, for cross-level gated and channel-grouped. It makes two alterations to the weighting step and leaves resizing and channel reduction untouched. Instead of three separate 1 by 1 convolutions each looking at one level, the three reduced feature maps are concatenated and passed through a single gate, so every weight is computed with the other levels visible. The gate then produces 24 channels rather than 3. These are read as three levels by eight groups, the softmax is taken across levels within each group, and each group's weight is broadcast over its share of the channels. The intention was to let channels that respond to texture draw from P3 while channels that respond to colour draw from P5 at the same pixel. Both ideas are taken from SKNet (Li, Wang, et al., 2019), which computes attention from a

joint descriptor of all branches and emits a per-channel softmax across them; we apply the same two properties across pyramid levels instead of kernel branches. Attentional Feature Fusion (Dai et al., 2021) makes a related argument that fusion weights should depend on content. The module adds roughly 7,000 parameters per fusion head, about 20,000 in total, which is under one percent of the model.

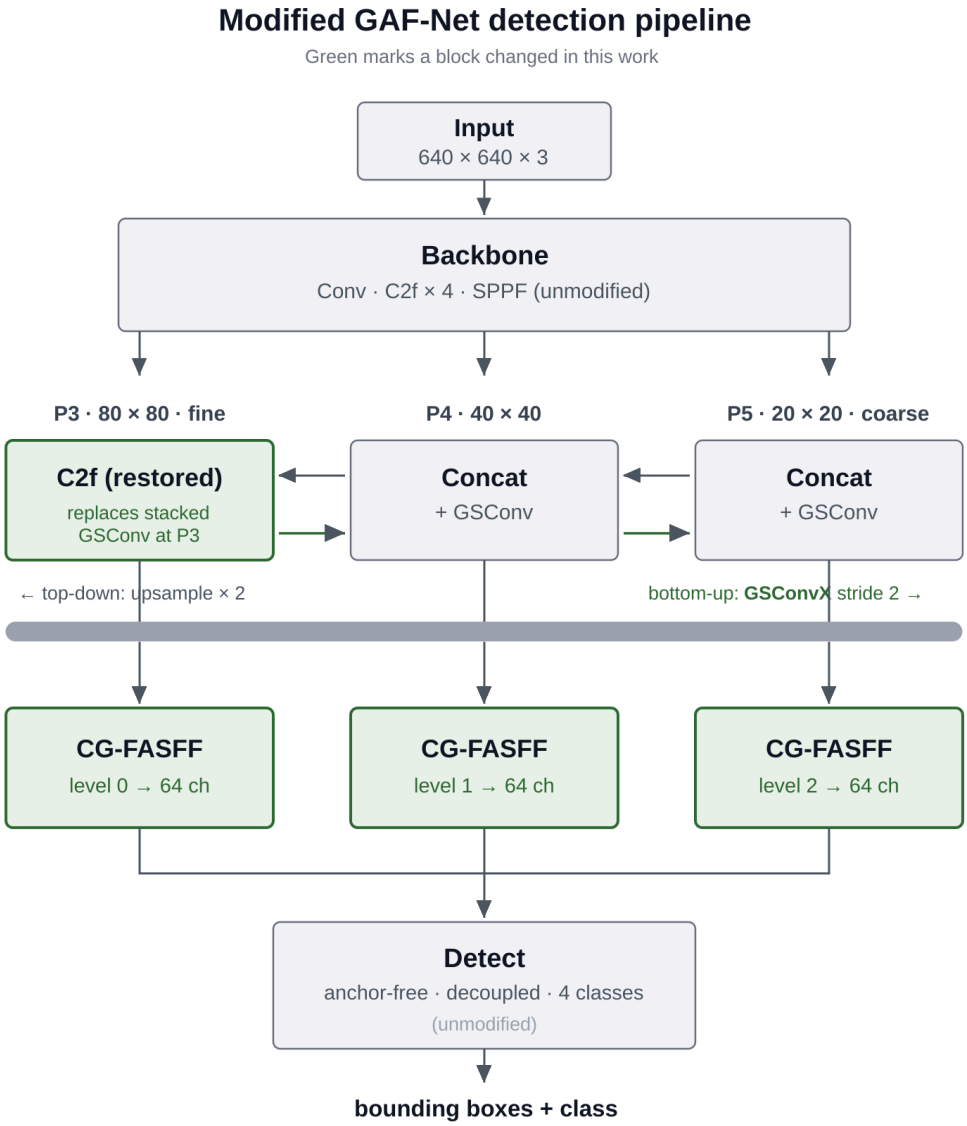


Figure 1. Overall architecture. The backbone and detection head follow stock YOLOv8. Both modified blocks sit in the neck.

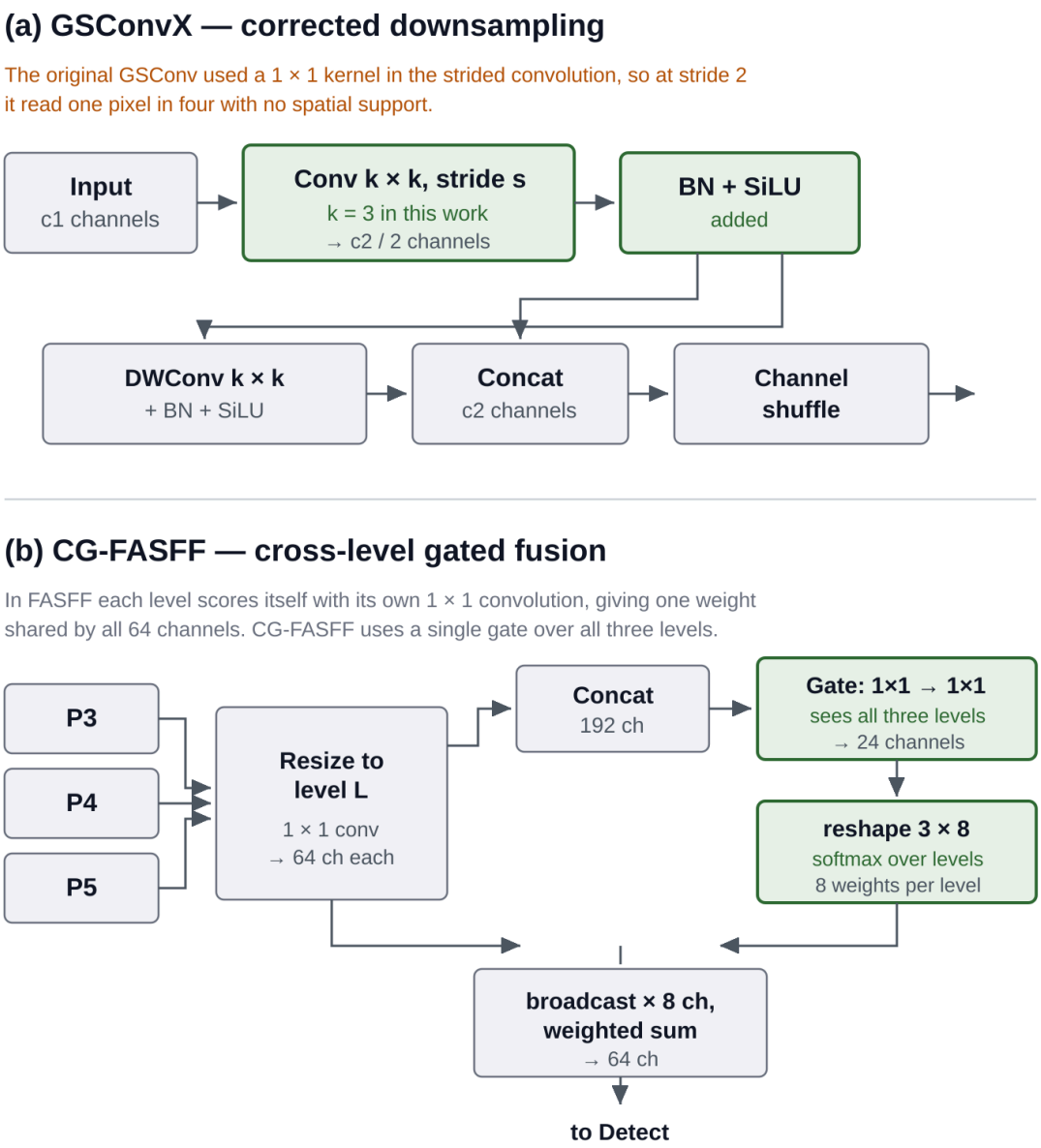


Figure 2. Internals of the two modified blocks. (a) GSConvX, with the corrected strided convolution. (b) CG-FASFF, with a single gate over the concatenated levels and eight weights per level.

Ablation design

To separate the contribution of each change we trained four variants. Model A is stock YOLOv8n. Model B is GAF-Net as published, with GSConv throughout the neck and FASFF at the three fusion heads. Model C adds our corrected downsampling and the restored C2f. Model D adds CG-FASFF on top of C. Every other setting is held fixed, so the only variable between rows is architecture.

Model	Neck convolution	P3 block	Fusion	Parameters	GFLOPs
A YOLOv8n	Conv	C2f	none	3.01 M	8.1
B GAF-Net	GSConv	GSConv	FASFF	1.93 M	6.5
C + GSConvX	GSConvX	C2f	FASFF	2.73 M	8.2
D + CG-FASFF	GSConvX	C2f	CG-FASFF	2.75 M	8.3

Table 1. The four variants compared in this study.

Dataset Details

We used the publicly available Corn Leaf Disease dataset hosted on Roboflow Universe, version 17, exported in YOLOv8 format. It contains 10,884 images across four classes: northern leaf blight, common rust, gray leaf spot and healthy leaves. The split is fixed by the publisher and we did not alter it.

Split	Images	Instances	Purpose
train	10,048	10,048	weight updates
valid	419	420	selects the best checkpoint each epoch
test	417	419	reported once, at the end
Total	10,884	10,887	

Table 2. Dataset splits.

The number of training images is not printed directly by Ultralytics, so we read it from the training log: 628 iterations per epoch at a batch size of 16. On the test split the four classes are unevenly represented, with 134 blight instances, 132 common rust, 53 gray spot and 100 healthy. Gray spot is both the visually hardest class and by some distance the smallest. With only 53 instances, a single missed detection moves its average precision by roughly 1.9 points, and we have been careful not to read too much into small differences on that column.

One structural property of this dataset shaped the entire project. The test split contains 417 images and 419 labelled instances, which works out to almost exactly one object per image, and the bounding box typically covers most of the leaf. The dataset used in the base paper is very different: 1,219 images carrying 7,460 instances, or roughly six objects per image at a range of sizes. Our set is nearly nine times larger in image count but far simpler in structure.

Property	GAF-Net paper	This study
Images	1,219	10,884
Labelled instances	7,460	10,884
Instances per image	6.1	1.0
Baseline mAP@0.5	76.7%	98.3%
Headroom above baseline	23.3 points	1.7 points

Table 3. Structural comparison with the dataset used in the base paper.

Two consequences follow. A gain of the size reported in the base paper is arithmetically unavailable here, because only 1.7 points exist between our baseline and a perfect score. And multi-scale fusion has very little to do, since all three pyramid levels are looking at the same single large object rather than at lesions of differing sizes. We return to both points when discussing why CG-FASFF failed.

Results and Visualizations

Training configuration

Every model was trained for 100 epochs at 640 by 640 resolution with a batch size of 16, a fixed seed of 0 and deterministic kernels enabled, on a single NVIDIA Tesla T4. We deliberately left the optimizer, learning rate, momentum and weight decay at their Ultralytics defaults. The optimizer setting therefore stayed on “auto”, which selects between SGD and AdamW based on the total number of update steps and sets the learning rate accordingly. Because dataset size, batch size, epoch count and class count are identical across all four runs, the automatic choice resolves identically for all of them. The loss is the standard YOLOv8 combination of CIoU box loss, binary cross-entropy classification loss and distribution focal loss, weighted 7.5, 0.5 and 1.5. Augmentation was left at defaults as well, including mosaic, which is disabled for the final ten epochs. Each run took approximately 2.66 hours, and the three new runs were completed in a single committed Kaggle session.

An uncontrolled variable in the first comparison

Our first attempt produced a result we could not explain. GAF-Net scored 0.9769 mAP@0.5 against the baseline's 0.9833, and gray spot average precision fell from 0.9630 to 0.9400. We spent some time looking for an architectural cause and formed the aliasing hypothesis described earlier. Before training anything further we compared the two runs' saved arguments, and they did not match. The baseline had been trained with the optimizer left on “auto”, while the GAF-Net run passed SGD with an explicit learning rate. The two results.csv files confirm the difference: the baseline logged eight learning-rate columns and the GAF-Net run logged three.

Retraining GAF-Net under the matched recipe, with no change whatsoever to the architecture, lifted gray spot average precision from 0.9400 back to 0.9630 and mAP@0.5 from 0.9769 to 0.9836. The deficit we had attributed to a downsampling bug was caused by an unexamined argument in a function call. We report this because it is the most transferable thing we learned in the project.

Results on the test split

Model	P	R	mAP@0.5	mAP@0.5:0.95	Params	GFLOPs
A YOLOv8n	.9711	.9643	.9833	.9042	3.01 M	8.1
B GAF-Net	.9612	.9483	.9836	.9115	1.93 M	6.5
C + GSConvX	.9670	.9468	.9854	.9053	2.73 M	8.2
D + CG-FASFF	.9837	.9290	.9794	.9047	2.75 M	8.3

Table 4. Detection results on the held-out test split (417 images, 419 instances, single seed).

Model	blight	common rust	gray spot	healthy
A YOLOv8n	.9862	.9891	.9630	.9950
B GAF-Net	.9850	.9915	.9630	.9950
C + GSConvX	.9872	.9921	.9674	.9950
D + CG-FASFF	.9833	.9889	.9504	.9950

Table 5. Per-class average precision at IoU 0.5.

Discussion

The clearest result is model B. GAF-Net matches the baseline on mAP@0.5, improves mAP@0.5:0.95 by 0.73 points, and does so using 1.08 million fewer parameters and 1.6 fewer GFLOPs. A 36% reduction in model size is far too large to be an artefact of initialisation, and it points the method towards the edge deployment scenario the original authors had in mind. On this dataset GAF-Net is better understood as an efficiency result than as an accuracy result.

Model C gives the best mAP@0.5 of the four and the best average precision on blight, common rust and gray spot. We are not claiming that as a finding. The margin over the baseline is 0.21 points on a single seed, against a total spread of 0.6 points across all four models, and we cannot separate an effect that small from the variation one would see between two runs of the same model with different random seeds.

Model D, our own proposed module, performed worst. It has the lowest mAP@0.5 and the lowest gray spot average precision, and the shape of the failure is informative. Its precision is the highest of the four at 0.9837 while its recall is the lowest at 0.9290. The model did not become less capable; it became more conservative, firing less often and

more confidently. We offer three explanations, in descending order of confidence. First, there is no multi-scale problem for the module to solve on this data. With one object per image filling most of the frame, all three pyramid levels describe the same thing, and the premise that different channel groups want different scales requires objects at different scales to exist. Second, widening the gate from three weights per pixel to twenty-four gives it the freedom to commit heavily to one level per group, and when the levels carry redundant information the uniform averaging it replaced was quietly acting as a useful ensemble. Third, the additional parameters were added to a model already sitting at the ceiling of what the dataset can measure.

That ceiling is the broader limitation. With a baseline at 98.3% mAP@0.5 there are 1.7 points available, and the four models span 0.6 of them. The dataset cannot rank these architectures, and we think reporting that honestly is more useful than presenting a two-tenths-of-a-point difference as an improvement.

Limitations

- **Single seed.** Every number comes from one training run. Differences below roughly half a point cannot be separated from initialisation variance. Three seeds per model with mean and standard deviation is the correct fix and would cost about 24 GPU-hours.
- **Saturated benchmark.** A harder or multi-instance corn dataset is needed before the fusion modules can be evaluated for what they were designed to do.
- **Small class support.** Gray spot carries 53 test instances. We use it as a diagnostic signal rather than as a headline number.
- **No robustness evaluation.** The base paper names occlusion and lighting as open problems and does not measure them. Neither did we. Corrupting the test set and re-running validation needs no retraining and is the obvious next experiment.
- **Partial transfer of pretrained weights.** Loading yolov8n.pt into a modified architecture only transfers layers whose shapes match, so models B, C and D began with partly random necks while model A did not. This favours model A and therefore makes B's efficiency result stronger rather than weaker, but we should flag the asymmetry rather than leave it buried.

Implementation and Dataset Links [GitHub]

The full implementation, including the custom module definitions, the modified architecture YAML files, the training notebook and the saved checkpoints, is available in the project repository. The notebook runs end to end on a Kaggle session with a single T4 GPU and downloads the dataset automatically through the Roboflow API.

- **Code repository:** https://github.com/Manideep-21/GAF-NET

- **Dataset:** https://universe.roboflow.com/aziz-jda2k/corn-leaf-disease-hgosu (version 17, YOLOv8 export)
- **Base paper:** https://doi.org/10.1109/TAFE.2025.3561196

Because custom modules cannot be registered with Ultralytics directly, the notebook patches parse_model. Each custom layer is temporarily replaced by a standard Conv with the same arguments so that Ultralytics performs its own channel and depth bookkeeping, and the real module is then built from the resolved channel counts and swapped in. The three fusion inputs are added to the save list so that their outputs are retained for later layers.

References

Cao, K., Yang, Y., Xu, L., Wan, X., Yang, L., & Jia, W. (2025). GAF-Net: An enhanced multidisease for corn leaves detection model based on YOLOv8. IEEE Transactions on AgriFood Electronics, 3(2), 420–432. https://doi.org/10.1109/TAFE.2025.3561196

Dai, Y., Gieseke, F., Oehmcke, S., Wu, Y., & Barnard, K. (2021). Attentional feature fusion. Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 3560–3569. https://arxiv.org/abs/2009.14082

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0) [Computer software]. https://github.com/ultralytics/ultralytics

Li, H., Li, J., Wei, H., Liu, Z., Zhan, Z., & Ren, Q. (2024). Slim-neck by GSConv: A lightweight-design for real-time detector architectures. Journal of Real-Time Image Processing, 21(3), Article 62.

Li, X., Wang, W., Hu, X., & Yang, J. (2019). Selective kernel networks. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 510–519. https://arxiv.org/abs/1903.06586

Liu, S., Huang, D., & Wang, Y. (2019). Learning spatial fusion for single-shot object detection. arXiv. https://arxiv.org/abs/1911.09516

Wang, C.-Y., Liao, H.-Y. M., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., & Yeh, I.-H. (2020). CSPNet: A new backbone that can enhance learning capability of CNN. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 390–391.

Zhang, R. (2019). Making convolutional networks shift-invariant again. Proceedings of the 36th International Conference on Machine Learning, 7324–7334. https://arxiv.org/abs/1904.11486